

DAY 13 — GENERIC REASONING + TOOL ORCHESTRATION + CONTEXT INTELLIGENCE

✓ GOAL

Move from:

retrieval + memory ✓

to:

generic engineering intelligence system ✓

WITHOUT:

- hardcoded thresholds ✗
 - metric-specific logic ✗
 - temperature rules ✗
 - domain assumptions ✗
-

✓ CORE IDEA OF NEW DAY 13

Python should NOT do engineering reasoning.

Instead:

Python → orchestration ✓

LLM → reasoning ✓

✓ WHAT DAY 13 ADDS

- ✓ structured context intelligence
- ✓ dynamic reasoning prompts
- ✓ tool orchestration
- ✓ generic engineering analysis
- ✓ context compression
- ✓ deduplication
- ✓ insight generation
- ✓ recommendation generation

WITHOUT hardcoding.

✅ DAY 13 ARCHITECTURE

User Query

↓

Conversation Memory ✅

↓

QueryRetriever ✅ Query Rewriter ✅

↓

Context Deduplicator ✅

↓

Structured Context Builder ✅

↓

Reasoning Prompt Generator ✅

↓

Qwen2.5 ✅

↓

✅ IMPORTANT SHIFT

OLD DAY 13:

Python decides anomaly ❌

NEW DAY 13:

LLM infers anomaly ✅

✅ TEMPLATE STRUCTURE

app/

└─ context/

| └─ context_builder.py ✅

| |— context_deduplicator.py ✓

| |— reasoning_prompt.py ✓

|

|— agent/

| |— ask.py ✓ MODIFY

✓ STEP 1 — CONTEXT DEDuplicATOR

 **app/context/context_deduplicator.py**

```
def deduplicate_context(results):
```

```
    unique = []
```

```
    seen = set()
```

```
    for r in results:
```

```
        if isinstance(r, dict):
```

```
            text = str(
```

```
                r.get("document", "")
```

```
            ).strip()
```

```
        else:
```

```
            text = str(r).strip()
```

```
            # ✓ skip duplicates
```

```
            if text in seen:
```

continue

seen.add(text)

unique.append(text)

return unique

✓ STEP 2 — STRUCTURED CONTEXT BUILDER

 **app/context/context_builder.py**

```
def build_structured_context(chunks):
```

```
    sections = []
```

```
    for idx, chunk in enumerate(chunks):
```

```
        block = f"""
```

```
[Context {idx+1}]
```

```
{chunk}
```

```
"""
```

```
    sections.append(block)
```

```
    return "\n".join(sections)
```

✓ STEP 3 — REASONING PROMPT GENERATOR

 **app/context/reasoning_prompt.py**

```
def build_reasoning_prompt(
```

```
    query,
```

```
    context
```

```
):
```

```
    prompt = f"""
```

You are an automotive engineering copilot.

Use ONLY the provided context.

Perform:

- engineering reasoning
- comparisons
- observations
- trend analysis
- recommendations

DO NOT use external knowledge.

If information is unavailable,

say:

"Answer not found in context."

=====

ENGINEERING CONTEXT

=====

{context}

=====

QUESTION

=====

{query}

=====

RESPONSE FORMAT

=====

1. Direct Answer
2. Engineering Observations
3. Comparative Insights
4. Recommendations

"""

return prompt

` `

STEP 4 — MODIFY ask.py

 **app/agent/ask.py**

```
import ollama
```

```
from app.agent.graph import (
```

```
run_agent
)

from app.memory.session_memory import (
    add_message
)

from app.memory.query_rewriter import (
    rewrite_query
)

from app.context.context_deduplicator import (
    deduplicate_context
)

from app.context.context_builder import (
    build_structured_context
)

from app.context.reasoning_prompt import (
    build_reasoning_prompt
)

def ask_question(query):

    # ✅ store query
    add_message(
        "user",
        query
```

)

 rewrite using memory

enhanced_query = rewrite_query(

 query

)

 retrieval

tool_output = run_agent(

 enhanced_query

)

 deduplicate

cleaned_chunks = deduplicate_context(

 tool_output["results"]

)

 structured context

context = build_structured_context(

 cleaned_chunks

)

 reasoning prompt

prompt = build_reasoning_prompt(

 query,

 context

)

```
response = ollama.chat(

    model="qwen2.5",

    messages=[

        {

            "role": "user",

            "content": prompt

        }

    ]

)

answer = response["message"]["content"]

# ✅ save response
add_message(

    "assistant",

    answer

)

return answer
```

✅ WHAT THIS NEW DAY 13 DOES

Instead of:

if value > threshold

you now do:

LLM-based reasoning ✅

using:

- retrieved context

- comparisons
 - observations
 - engineering interpretation
-

✅ EXAMPLE

USER

Compare front pad and rear liner temperature

CONTEXT SENT TO QWEN

Front pad:

163 | 179 | 188

Rear liner:

153 | 153 | 146

QWEN REASONS

Front pad temperatures are consistently higher.

Possible braking system. Possible higher thermal loading observed

Review brake balance and cooling performance.

- ✅ NO hardcoded rules
 - ✅ Generic reasoning
 - ✅ Works for any engineering metric
-

✅ WHY THIS IS BETTER

This architecture is now:

- ✅ generic
- ✅ scalable

- ✓ multi-domain
- ✓ future-proof
- ✓ LLM-native
- ✓ retrieval-grounded

..

instead of:

temperature-specific Python rules ✗

✓ IMPORTANT DESIGN PRINCIPLE

✓ Python Responsibilities

- ✓ retrieval
- ✓ orchestration
- ✓ memory
- ✓ chunking
- ✓ storage
- ✓ context building
- ✓ tool execution

✓ LLM Responsibilities

- ✓ engineering reasoning
- ✓ comparisons
- ✓ observations
- ✓ recommendations
- ✓ interpretation
- ✓ analysis

✓ CURRENT SYSTEM

PDF



Extraction



Normalization



Filtering



Embeddings



ChromaDB



DuckDB



Hybrid Retrieval



Conversation Memory



Query Rewriter



Context Intelligence 



LLM Reasoning 



Engineering Copilot 

 **YOU ARE NOW HERE**

 DAY 13 COMPLETE

Next:

DAY 14

- autonomous engineering workflows
- multi-document reasoning
- report comparison
- dynamic tool planning
- engineering dashboard